

Sri Lanka Institute of Information Technology



Lab Submission
Lab - 05

IT25102546

Fonseka D.V.A.N

Artificial Intelligence and Machine Learning | IT2011

B.Sc. (Hons) in Information Technology

Exercise 2 – LLM Leaderboard

My Favorite LLM: GPT-5.6 Sol (max)

Current Position: 5th overall among the current models displayed in the intelligence-sorted leaderboard.

Artificial Analysis Intelligence Index: 61

Date Checked: 23 August 2026

Reason: GPT-5.6 Sol (max) is my favorite LLM because it provides intelligent, detailed, and useful responses for academic and technical tasks.

Filter, e.g. GPT, Meta

Expand columns

Weights: AllSize: AllPrice: AllReasoning: AllStatus: CurrentReasoning model

Model	Features		Intelligence		Price	Speed	Latency	End-to-End Response Time		Further Analysis	
	Context Window	Creator	Artificial Analysis Intelligence Index	Cost per Task USD	Median Tokens/s	Latency First Chunk (s)	Total Response (s)				
Claude Opus 5 (max)	1M	Anthropic	63	\$2.34	56	40.66	49.65	Model	Providers		
Claude Opus 5 (xhigh)	1M	Anthropic	63	\$1.80	52	29.84	39.41	Model	Providers		
Claude Fable 5 (with fallback)	1M	Anthropic	62	\$3.14	69	97.83	105.03	Model	Providers		
Claude Opus 5 (high)	1M	Anthropic	61	\$1.23	53	15.85	25.37	Model	Providers		
GPT-5.6 Sol (max)	1M	OpenAI	61	\$1.23	75	104.85	111.54	Model	Providers		
Grok 4.6 (high)	500k	SpaceXAI	61	\$0.84	67	43.66	51.15	Model	Providers		
Grok 4.6 (xhigh)	500k	SpaceXAI	60	\$1.04	65	46.08	53.80	Model	Providers		
Kimi K3 (max)	1.05M	Kimi	60	\$0.84	35	3.30	74.48	Model	Providers		
GLM-5.3 (max)	1M	Z AI	60	\$0.68	--	--	--	Model	Providers		
GPT-5.6 Sol (xhigh)	1M	OpenAI	59	\$0.81	69	26.79	34.01	Model	Providers		
Grok 4.6 (medium)	500k	SpaceXAI	59	\$0.67	71	27.33	34.37	Model	Providers		
Claude Opus 5 (medium)	1M	Anthropic	59	\$0.72	53	4.92	14.30	Model	Providers		

The two models I had not tried before were Claude and Gemini.

Model 1: Claude

Prompt 1:

Create a simple one-week study plan for a second-year university student studying Artificial Intelligence and Machine Learning.

Response 1:

Here's a one-week study plan tailored for a second-year AI/ML student, balancing core theory (math, algorithms) with practical coding skills.

Weekly Study Plan – AI & Machine Learning (Year 2)

Daily study block: ~3–4 hours, split into focused sessions

Day	Focus Area	Activities
Monday	Math Foundations (Linear Algebra)	Review vectors, matrices, eigenvalues/eigenvectors. Solve 10–15 practice problems. Watch one lecture (e.g., 3Blue1Brown's Essence of Linear Algebra) if concepts feel rusty.
Tuesday	Probability & Statistics	Cover distributions, Bayes' theorem, expectation/variance. Apply concepts by coding a small Bayesian classifier in Python.
Wednesday	Core ML Algorithms	Study one algorithm in depth (e.g., logistic regression or decision trees) — derive the math, then implement it from scratch in Python (no libraries) to build intuition.
Thursday	Applied ML with Libraries	Use scikit-learn to solve a small dataset problem (e.g., Titanic or Iris dataset). Practice train/test splits, cross-validation, and evaluation metrics (accuracy, precision, recall, F1).
Friday	Neural Networks / Deep Learning	Study forward/backward propagation basics. Build a simple neural net (e.g., with NumPy or PyTorch) on a toy

Day	Focus Area	Activities
	Intro	dataset like MNIST digits.
Saturday	Project Work / Coding Practice	Work on a mini personal project or Kaggle competition. Alternatively, solve algorithm/data structure problems (important for technical interviews later).
Sunday	Review & Reading	Lighter day: review the week's notes, read one research paper or blog post (e.g., from Papers With Code or Distill.pub), and plan next week's topics.

Tips to Get the Most Out of This Plan

- **Alternate theory and code** each day so concepts don't stay abstract.
- **Keep a running notebook** (physical or Notion/Obsidian) summarizing key formulas and intuitions — this becomes invaluable before exams.
- **Build small projects** rather than only doing tutorials; implementing something end-to-end cements understanding far better than passive learning.
- **Track weak spots** — if a topic from Monday felt shaky, revisit it briefly on Sunday's review day.

Want me to adjust this based on your specific course syllabus, upcoming exams, or how many hours per day you actually have available?

Prompt 2: Explain overfitting in machine learning using a simple real-life example.

Response 2:

Overfitting in Machine Learning

Overfitting happens when a model learns the training data *too well* — including its noise and quirks — so it performs great on data it's seen but poorly on new, unseen data. It's essentially memorizing instead of learning general patterns.

Real-life example: Studying for an exam

Imagine a student preparing for a math test by memorizing the **exact answers** to 50 practice problems, instead of understanding the underlying method to solve them.

- **On the practice test:** They score 100%, because they've memorized every question and answer perfectly.
- **On the real exam:** The questions are *similar* but not identical (different numbers, slightly different wording). The student fails, because they never learned the actual concept — just the specific answers.

This student has "overfit" to the practice problems. They didn't generalize.

Compare that to a student who understands the *method* behind solving the problems. They might score slightly lower on the practice test (maybe 90%, since memorization can be faster/easier for known questions), but they do well on the real exam too, because they grasped the underlying pattern.

Mapping it to ML terms

Study Analogy	ML Term
Practice problems	Training data
Real exam	Test/unseen data
Memorizing answers	Overfitting
Understanding the method	Generalization
Student's failing score on the real exam	High test error despite low training error

Why it happens

- Too complex a model relative to the amount/simplicity of data (like memorizing instead of understanding)
- Training for too long
- Not enough training examples to learn general patterns

How it's prevented

- **More data** — harder to memorize, easier to spot true patterns
- **Simpler models** — fewer parameters to "memorize" with
- **Regularization** — a penalty that discourages the model from fitting noise
- **Cross-validation** — testing on held-out data during training to catch overfitting early
- **Early stopping** — stop training before the model starts memorizing noise

Would you like a visual (like a chart showing training vs. validation error curves) to see what overfitting looks like numerically?

Screenshots:

Prompt 1 & response

Create a simple one-week study plan for a second-year university student studying Artificial Intelligence and Machine Learning.

Here's a one-week study plan tailored for a second-year AI/ML student, balancing core theory (math, algorithms) with practical coding skills.

Weekly Study Plan - AI & Machine Learning (Year 2)

Daily study block: ~3-4 hours, split into focused sessions

Day	Focus Area	Activities
Monday	Math Foundations (Linear Algebra)	Review vectors, matrices, eigenvalues/eigenvectors. Solve 10-15 practice problems. Watch one lecture (e.g., 3Blue1Brown's Essence of Linear Algebra) if concepts feel rusty.
Tuesday	Probability & Statistics	Cover distributions, Bayes' theorem, expectation/variance. Apply concepts by coding a small Bayesian classifier in Python.
Wednesday	Core ML Algorithms	Study one algorithm in depth (e.g., logistic regression or decision trees) — derive the math, then implement it from scratch in Python (no libraries) to build intuition.
Thursday	Applied ML with Libraries	Use scikit-learn to solve a small dataset problem (e.g., Titanic or Iris dataset). Practice train/test splits, cross-validation, and evaluation metrics (accuracy, precision, recall, F1).
Friday	Neural Networks / Deep Learning Intro	Study forward/backward propagation basics. Build a simple neural net (e.g., with NumPy or PyTorch) on a toy dataset like MNIST digits.
Saturday	Project Work / Coding Practice	Work on a mini personal project or Kaggle competition. Alternatively, solve algorithm/data structure problems (important for technical interviews later).
Sunday	Review & Reading	Lighter day: review the week's notes, read one research paper or blog post (e.g., from Papers With Code or Distill.pub), and plan next week's topics.

Tips to Get the Most Out of This Plan

- Alternate theory and code each day so concepts don't stay abstract.
- Keep a running notebook (physical or Notion/Obsidian) summarizing key formulas and intuitions — this becomes invaluable before exams.
- Build small projects rather than only doing tutorials; implementing something end-to-end cements understanding far better than passive learning.
- Track weak spots — if a topic from Monday felt shaky, revisit it briefly on Sunday's review day.

Want me to adjust this based on your specific course syllabus, upcoming exams, or how many hours per day you actually have available?

Prompt 2 & response

Explain overfitting in machine learning using a simple real-life example

4 minutes ago

Overfitting in Machine Learning

Overfitting happens when a model learns the training data *too well* — including its noise and quirks — so it performs great on data it's seen but poorly on new, unseen data. It's essentially memorizing instead of learning general patterns.

Real-life example: Studying for an exam

Imagine a student preparing for a math test by memorizing the exact answers to 50 practice problems, instead of understanding the underlying method to solve them.

- **On the practice test:** They score 100%, because they've memorized every question and answer perfectly.
- **On the real exam:** The questions are *similar* but not identical (different numbers, slightly different wording). The student fails, because they never learned the actual concept — just the specific answers.

This student has "overfit" to the practice problems. They didn't generalize.

Compare that to a student who understands the *method* behind solving the problems. They might score slightly lower on the practice test (maybe 90%, since memorization can be faster/easier for known questions), but they do well on the real exam too, because they grasped the underlying pattern.

Mapping it to ML terms

Study Analogy	ML Term
Practice problems	Training data
Real exam	Test/unseen data
Memorizing answers	Overfitting
Understanding the method	Generalization
Student's falling score on the real exam	High test error despite low training error

Why it happens

- Too complex a model relative to the amount/simplicity of data (like memorizing instead of understanding)
- Training for too long
- Not enough training examples to learn general patterns

How it's prevented

- **More data** — harder to memorize, easier to spot true patterns
- **Simpler models** — fewer parameters to "memorize" with
- **Regularization** — a penalty that discourages the model from fitting noise
- **Cross-validation** — testing on held-out data during training to catch overfitting early
- **Early stopping** — stop training before the model starts memorizing noise

Would you like a visual (like a chart showing training vs. validation error curves) to see what overfitting looks like numerically?

Model 2: Gemini

Prompt 1:

Create a simple one-week study plan for a second-year university student studying Artificial Intelligence and Machine Learning.

Response 1:

This balanced 7-day study plan distributes focus across core second-year AIML subjects—linear algebra, probability, machine learning algorithms, and deep learning basics—while protecting time for coding practice and rest.

Weekly Overview

Day	Primary Focus	Key Topics	Practical Task
Monday	Linear Algebra	Matrix operations, Eigenvalues, Eigenvectors	Implement matrix multiplication from scratch in NumPy
Tuesday	Probability & Stats	Bayes' Theorem, Probability distributions	Solve 5 conditional probability problems
Wednesday	Core ML Models	Linear & Logistic Regression,	Build Linear Regression using scikit-

D a y	Prima ry Focus	Key Topics	Practic al Task
		Gradie nt Descen t	learn on a sample dataset
T h u	Data Proces sing	Data cleanin g, Feature scaling, EDA	Load a Kaggle dataset using pandas and plot distribut ions
F r i	Deep Learni ng Intro	Neural Networ k foundat ions, Activat ion functio ns	Code a single perceptr on or simple feedfor ward net in PyTorc h
S a t	Projec t & Revie w	Hands- on applica tion, Mini- project work	Connect data cleaning + ML model pipeline in a Jupyter Notebo ok
S u	Rest & Reset	Active recover y,	Review key formula

D a y	Prima ry Focus	Key Topics	Practic al Task
n		Light recap	cheat sheets for 30 mins; rest for the week

Daily Execution Structure

- **Block 1 (90 mins):** Theory & Core Concepts (Readings, lecture slides, note-taking)
- **Block 2 (90 mins):** Practical Application (Coding in Python, implementing algorithms)
- **Block 3 (45 mins):** Problem Solving (Problem sets, past exam questions)

Study Tips for AIML Majors

- **Don't skip the math:** ML libraries hide math behind single function calls, but exam questions and debugging require understanding vector transformations and gradients.
- **Keep a central code repository:** Commit your daily practice code to a GitHub repository to build a portfolio as you learn.
- **Active recall over re-reading:** Use flashcards or self-quizzing for key loss functions, matrix properties, and optimization algorithms.

Prompt 2:

Explain overfitting in machine learning using a simple real-life example.

Response 2:

Overfitting happens when a model learns its training data **too well**—memorizing specific noise and quirks rather than underlying patterns. As a result, it performs perfectly on familiar data but fails when given new, unseen data.

The Real-Life Analogy: Studying for a Math Exam

Imagine two students preparing for a math test:

- **Student A (Good Fit):** Learns the core formulas and underlying logic. When given a problem on the exam with completely new numbers, they easily solve it because they understand the *concept*.
- **Student B (Overfitting):** Memorizes every exact practice problem, number for number, down to the font size and page margins.
 - On the practice test, Student B scores **100%**.
 - On the actual exam, the teacher changes the numbers slightly. Student B panics and fails because they only memorized specific answers, not the rules.

How This Translates to Machine Learning

In data terms, an overfitted model draws an overly complex boundary around its training set to capture every single data point—including outliers.

Concept	Machine Learning Context	Real-Life Analogy
Training Data	Data used to train the algorithm	Practice test questions
Testing Data	New, unseen data used to evaluate performance	The actual exam
Low Bias / High Variance	Memorizes training data noise, fails to generalize	Knows practice questions by heart, gets confused by new questions
The Goal	Generalization to new data	True understanding of the subject

Common Ways to Fix Overfitting:

- **Simplify the model:** Reduce the number of features or tree depth.
- **Get more data:** Exposing the model to more examples makes it harder to memorize individual points.
- **Regularization:** Add a penalty for overly complex decisions.
- **Cross-Validation:** Split data into multiple train/test sets to catch memorization early.

Screenshots:

Prompt 1 & response

Create a simple one-week study plan for a second-year university student studying Artificial Intelligence and Machine Learning.

This balanced 7-day study plan distributes focus across core second-year AIML subjects—linear algebra, probability, machine learning algorithms, and deep learning basics—while protecting time for coding practice and rest.

Weekly Overview

Day	Primary Focus	Key Topics	Practical Task
Mon	Linear Algebra	Matrix operations, Eigenvalues, Eigenvectors	Implement matrix multiplication from scratch in NumPy
Tue	Probability & Stats	Bayes' Theorem, Probability distributions	Solve 5 conditional probability problems
Wed	Core ML Models	Linear & Logistic Regression, Gradient Descent	Build Linear Regression model using <code>scikit-learn</code> on a sample dataset

Thu	Data Processing	Data cleaning, Feature scaling, EDA	Load a Kaggle using pandas plot distribution
Fri	Deep Learning Intro	Neural Network foundations, Activation functions	Code a single perceptron or feedforward r PyTorch
Sat	Project & Review	Hands-on application, Mini-project work	Connect data + ML model pi a Jupyter Not
Sun	Rest & Reset	Active recovery, Light recap	Review key fo cheat sheets l mins; rest for

...

Daily Execution Structure

- **Block 1 (90 mins):** Theory & Core Concepts (Readings, lecture slides, note-taking)
- **Block 2 (90 mins):** Practical Application (Coding in Python, implementing algorithms)
- **Block 3 (45 mins):** Problem Solving (Problem sets, past exam questions)

Study Tips for AIML Majors

- **Don't skip the math:** ML libraries hide math behind single function calls, but exam questions and debugging require understanding vector transformations and gradients.
- **Keep a central code repository:** Commit your daily practice code to a GitHub repository to build a portfolio as you learn.
- **Active recall over re-reading:** Use flashcards or self-quizzing for key loss functions, matrix properties, and optimization algorithms.

Want me to tailor this schedule around your specific course list or exam dates?

↪ Yes

Prompt 2 & response

Explain overfitting in machine learning using a simple real-life example

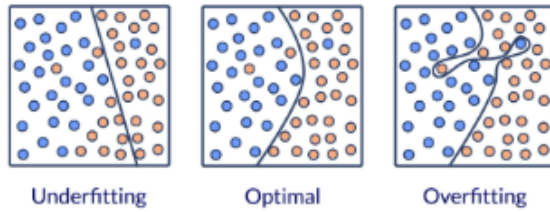


Overfitting happens when a model learns its training data **too well**—memorizing specific noise and quirks rather than underlying patterns. As a result, it performs perfectly on familiar data but fails when given new, unseen data.

The Real-Life Analogy: Studying for a Math Exam

Imagine two students preparing for a math test:

- **Student A (Good Fit):** Learns the core formulas and underlying logic. When given a problem on the exam with completely new numbers, they easily solve it because they understand the *concept*.
- **Student B (Overfitting):** Memorizes every exact practice problem, number for number, down to the font size and page margins.
 - On the practice test, Student B scores **100%**.
 - On the actual exam, the teacher changes the numbers slightly. Student B panics and fails because they only memorized specific answers, not the rules.



Underfitting vs. Optimal vs. Overfitting. Source: Tokuma Ipuna / Getty Images

How This Translates to Machine Learning

In data terms, an overfitted model draws an overly complex boundary around its training set to capture every single data point—including outliers.

Concept	Machine Learning Context	Real-Life Analogy
Training Data	Data used to train the algorithm	Practice test questions
Testing Data	New, unseen data used to evaluate performance	The actual exam

Testing Data	New, unseen data used to evaluate performance	The actual exam
Low Bias / High Variance	Memorizes training data noise, fails to generalize	Knows practice questions by heart, gets confused by new questions
The Goal	Generalization to new data	True understanding of the subject

...

Common Ways to Fix Overfitting:

- **Simplify the model:** Reduce the number of features or tree depth.
- **Get more data:** Exposing the model to more examples makes it harder to memorize individual points.
- **Regularization:** Add a penalty for overly complex decisions.
- **Cross-Validation:** Split data into multiple train/test sets to catch memorization early.

Want to see code examples of regularization techniques like L1/L2 or Dropout?

Yes

- Gemini gave a more detailed response.
- Claude gave a shorter and easier-to-understand response.
- In my opinion, Gemini produced the better response because it was more detailed and well structured.